

Lecture 5

Logical Agents

Intelligent Systems
Vrije Universiteit

1

Today we will be looking at how we can use logic to express and manipulate information, most importantly in the cases of imperfect information.

Practical how-to / best practices

1 Practicalities

2 Expectation Management

Tasks are often more simple than you think

If you fail with a subtask, explain what you did

3 Best Practices / Guidelines

Use Slack channels

Answer each others questions

4 Using of external sources

Googling is good

5 Chat GPT and Plagiarism

Overview

- 1 Logical Reasoning over Game States
- 2 Logics & Formal Systems
- 3 Propositional Logic
- 4 SAT Solving

last weeks Part 6: Playing Schnapsen

Intelligent Systems
Vrije Universiteit

Schnapsen *Phase 1*

In Phase 1, Schnapsen is **deterministic** because there is no element of chance once all cards are arranged.

We have **imperfect information** because there are many possible card deals.

The best strategy: assume perfect information and average

- Play minimax for all possible deals and choose the one most likely to win

$$\operatorname{argmax} \sum P(s) \operatorname{minimax}(\operatorname{result}(s,a))$$

Schnapsen Phase 1 is challenged by the fact that it deals with imperfect information. The agent can not see every card in the deck.

The best strategy to win nevertheless is to use the minimax algorithm to search through every possible deal that we can not observe. This way we know for which games we would win or lose. For all possible deals, we then choose the one which is most likely to win. We can do this by averaging over the possible outcomes of each minimax run.

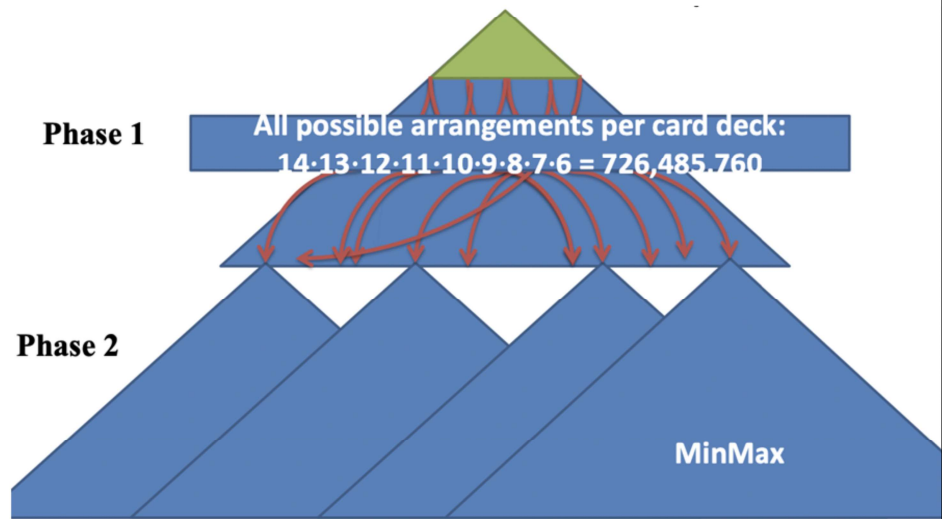
Minimax Phase 2

In total 10^{20} possibilities!

After trick 3 there are 336 possible deals. Minimax will take

$$5! * 5! * 5^2 * 5^2 = 9,000,000$$

$$\text{Complexity} = 3 * 10^9$$



Phase 2 of Schnapsen also results in a number of possible arrangements that is impossible for a computer to calculate with minimax.

Minimax *Phase 1*

At the beginning of...

Trick 5: maximum 6 deals

Trick 4: $8*7*6 = 336$

Trick 3: $10*9*8*7*6 = 30,240$

Trick 2: $12*11*10*9*8*7*6 =$
3,991,680

Trick 1 (the game): $14! / 5! =$
726,485,760

Minimax : $5! * 5! * 5^5 * 5^5 = 140,625,000,000$

7

Each player starts with 5 cards in their hand, not visible to the opponent. In trick 5 there are 6 possible deals because there are two cards left in the talon (undealt deck of cards), one of which is the trump card. So there are 6 different possibilities for how the cards are distributed.

When we continue to perform such combinatorics on all the tricks in phase 1, we realize the impossibility of performing minimax on every possible deal.

Perfect Information Monte Carlo Sampling (PIMS)

PIMS is a sampling technique for playing imperfect information games. It randomly samples n times by playing n games.

$$\operatorname{argmax} (1/n) \sum_{i=1}^n \operatorname{minimax}(\operatorname{result}(s_i, a))$$

Still expensive with 5^{10} nodes to be explored. In the Schnapsen platform we will use, sampling is done with **random rollouts**; play a complete game with one player making random moves and use this to define your weights.

$$\operatorname{argmax} (1/n) \sum_{i=1}^n \operatorname{randomrollouts}(s_i, a)$$

Perfect Information Monte Carlo Sampling can be used to alleviate this computational problem. By sampling, meaning randomly taking a subset, and averaging over this subset, you can generalise the outcome to the total population of subsets. For example, we can choose $n = 1000$ games out of 700 million total possible games to play out using minimax. Although still computationally expensive, we significantly reduce complexity.

The **random rollout** method, used in Schnapsen platforms, plays out a random number n of games and simply checks whether we win or lose in the end.

AlphaGo



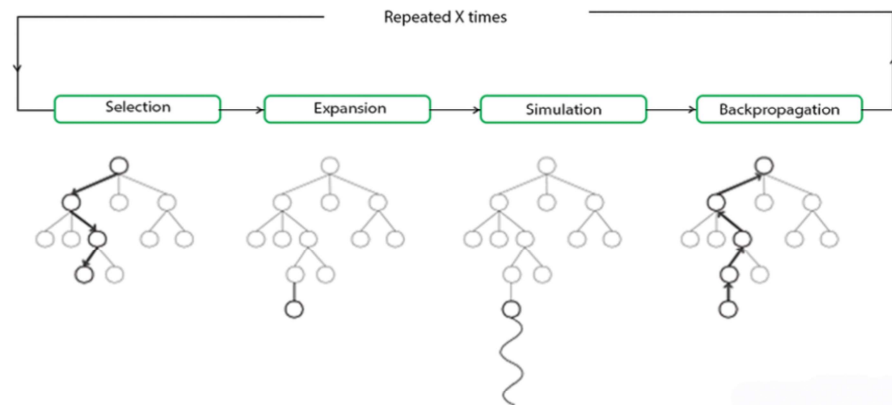
AlphaGo vs. Lee Sedol, 2016

Go is one of the world's oldest board games originating in East Asia, with more search choices at any given time than atoms in the universe. In 2016, Google DeepMind's AlphaGo beat Lee Sedol, one of the world's top Go players. How? By using a combination of alpha-beta pruning for the minimax algorithm and Monte Carlo tree search.

Monte Carlo Tree Search

Monte Carlo Tree Search is a heuristic search algorithm for deterministic games.

- It is the core of the AlphaGo approach, which learns the structure of the search tree



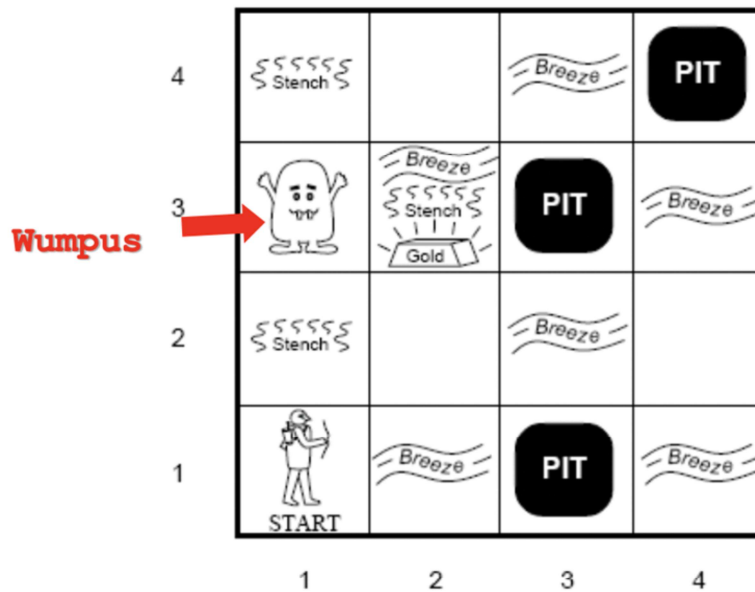
10

This random rollout approach was also used in Google's AlphaGo (see next slide) with **Monte Carlo tree search** to learn the structure of the search tree.

Part 1: Logical Reasoning over Game States

Intelligent Systems
Vrije Universiteit

Intro to Wumpus World



Link: [Wumpus World VR₁₂](#)

In Wumpus World, the agent is an explorer who starts in position 1,1 and whose goal is to reach the gold in some unknown square. Along the way, the agent must avoid falling into pits or coming across the Wumpus. We will look at a limited number of possible actions in order to evaluate the game.

Wumpus World *PEAS*

Recall **P**erformance

Environment

Actuators

Sensors

Performance measure

gold +1000, death -1000

-1 per step, -10 for using the arrow

Environment

Squares adjacent to wumpus are smelly

Squares adjacent to pit are breezy

Glitter iff gold is in the same square

Shooting kills wumpus if you are facing it

Shooting uses up the only arrow

Grabbing picks up gold if in same square

Releasing drops the gold in same square

Sensors Breeze, Glitter, Smell

Actuators Left turn, Right turn,

Forward, Grab, Release, Shoot

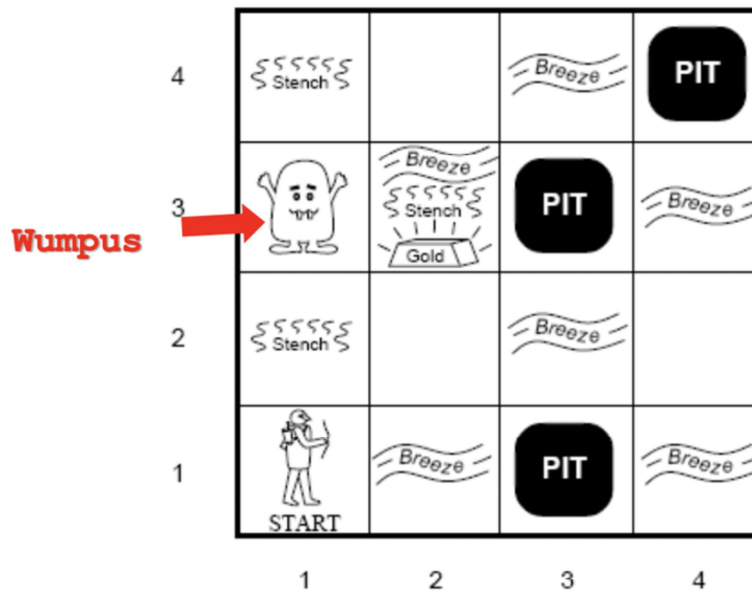
Recall the environment properties that we can assess using the *PEAS* acronym. In Wumpus World the agent only has partial knowledge, and can only move to an adjacent cell. If the agent comes across a stench then the Wumpus is in an adjacent cell, and if they come across a breeze the pit is in an adjacent cell.

Wumpus World *Environment Properties*

Observable	X	Only local perception
Deterministic	✓	Outcome of action predictable
Static	✓	Pits don't move
Discrete	✓	Finite set of actions
Single-agent	✓	Wumpus only agent
Complete Information	X	Partial information in each state

The properties of the environment are crucial to applying logic at a later step.

Perfect Information Wumpus World simplification



15

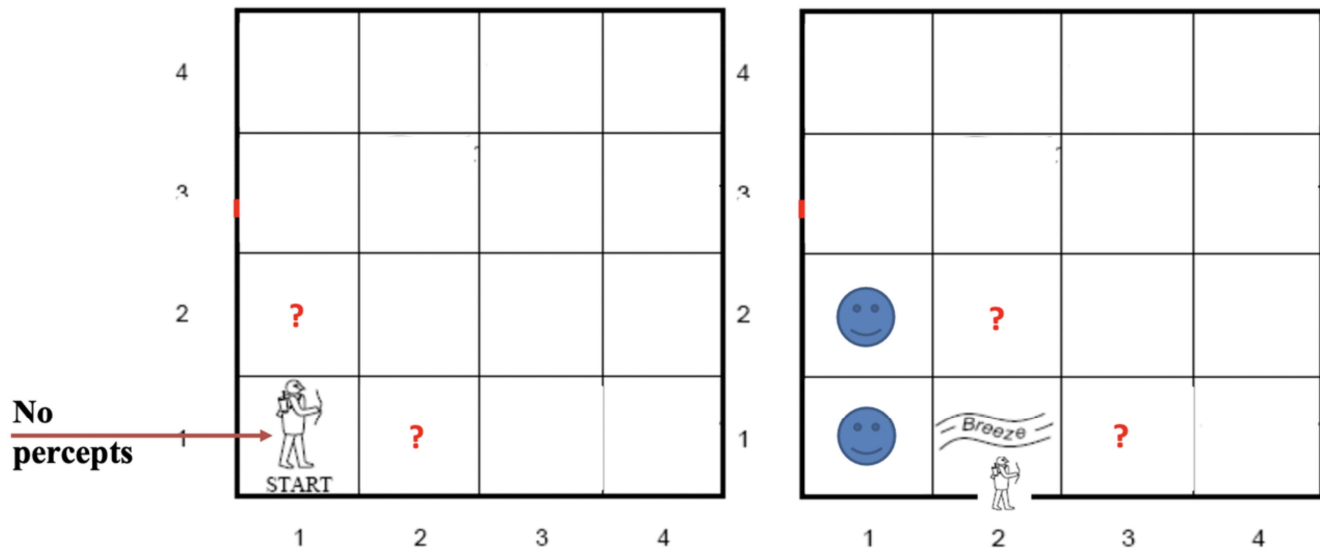
Let us first consider a variant of Wumpus with full observability. This can be represented as a State-Space Problem and solved by search.

Let us do this as an exercise.

However, we do not know which board-position we are in. There are 16 possible positions for the Wumpus, 16 for the Gold, and $16 \cdot 15 \cdot 14$ different ways to position 3 Pits on a 16x16 board. In total there are 860.160 different board positions.

We could use this with PIMS.

The Real Wumpus World



16

In the real Wumpus World, we do NOT know anything about the states we have not been yet. So we are in an imperfect

This is the starting position of the agent, with no knowledge about what is in the other cells.

At **1,1** we have no percepts, meaning no breeze and no smell. We can conclude the **1,2** and **2,1** are both safe cells, so we can randomly choose one to explore.

After moving to **2,1**, the agent feels a breeze, meaning that **2,2** or **3,2** have a pit. Thus, we move back to **1,1**.

Logic

Logic is used to represent an agent's environment in a declarative way.

Declarative means using explicit statements to say something about the state of the world.

We use logic in AI to

- Deal with partial or implicit information
- Knowledge about environment & task
- Reason over possible states
- Represent human knowledge

Logic can help us to solve complex problems, largely because we can represent human knowledge in a structured way. It is useful in playing games such as Schnapsen, because we can represent rules and states of the game using logic. Today we're looking at **declarative** logic, which are explicit statements, or declarations, about things that are true in the world.

Wumpus World Logical Exploration

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A OK	OK		

(a)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK	P?		
1,1	2,1	3,1	4,1
V OK	A B OK	P?	

(b)

Knowledge base (KB) initially contains only rules of the environment.

Position Percept

[1,1] First percept is *[none, none]*, move to safe cell e.g. [2,1]

[2,1] Breeze indicates that there is a pit in [2,2] or [3,1], return to [1,1], try next safe cell

18

We can represent these game states more specifically by using variables to start formalising the percepts that the agent has collected thus far.

All of these percepts are added to the agent's **knowledge base** or **KB**, which is a list of all things that the agent knows to be true.

Wumpus World Logical Exploration

Position	Percept
[1,2]	Stench indicates Wumpus in [1,3] or [2,2]. <u>BUT</u> not [2,2] or stench would have been detected in [2,1]. <u>THUS</u> Wumpus in [1,3], move to [2,2].
[2,2]	No indicators [2,3] or [3,2]. Move to [2,3].

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

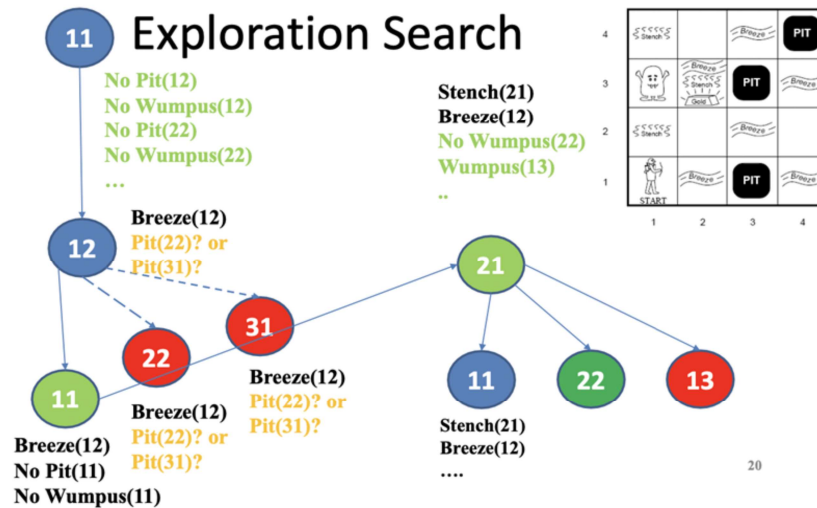
A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

The agent can continue to add percepts to the knowledge base by exploring the environment. In continuing the exploration, the agent moves from 1,1 to 2,1 and perceives a stench.

The agent can logically conclude that the Wumpus is in cell 1,3. The agent knows that 2,2 does not have a Wumpus because there was no stench in 2,1 where they were earlier.

Wumpus World as Exploration Search

$KB = \text{wumpus-world-rules} + \text{observations}$
 a_1 "[1,2] is safe", $KB \models a_1$, proved by logical reasoning



We can formalise the knowledge base as the conjunction of the Wumpus World rules and the percepts that the agent perceives using its sensors.
 This logical process of adding statements to our knowledge base is very similar to the search trees that we have already seen.

Part 2: Logics and Formal Systems

Intelligent Systems
Vrije Universiteit

21

Let's translate this kind of intuitive logic into formal systems. What can logic really do for us? Logic allows to model games and answer questions such as, 'Which cell is safe to move to?'

Knowledge Base

$$KB = \text{wumpus-world-rules} + \text{observations}$$

A **knowledge base** is a collection of statements, **propositions**, which we know to be true.

Wumpus World *Examples*

	Statement	Formalisation	Operator
Observation	"no breeze in [1,1]"	$\neg B_{1,1}$	$\neg$ Negation
Rule	"breeze in [1,1] if and only if pit [2,1] or [1,2]"	$B_{1,1} \text{ iff } P_{2,1} \text{ or } P_{1,2}$	or Disjunction

22

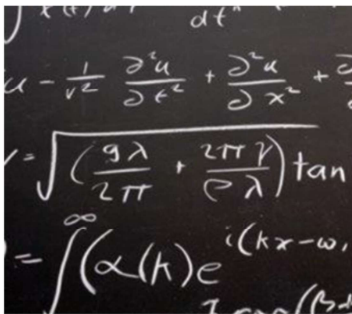
As we've just seen, the knowledge base is a collection of true statements. In Wumpus World, the percepts can be formalised by translating them into propositions.

For example, "no breeze in [1,1]" can be translated into $\neg B_{1,1}$, which uses the negation operator to say NOT. We can also use the disjunction operator to express the concept of OR. We will return to these operators later, but just these two operators can tell us a lot about an environment.

What We Need for a *Logic* or *Formal System*

Syntax

The rules or **language** used for constructing and transforming symbols and operators in propositions.

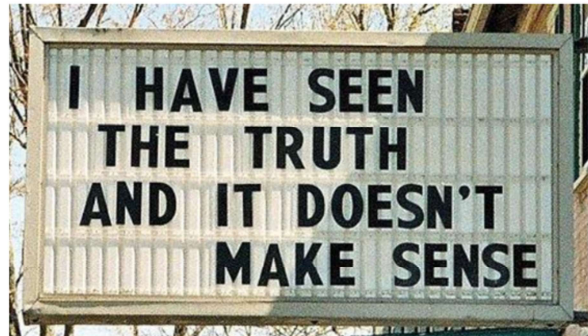


Handwritten mathematical formulas on a chalkboard, illustrating syntax. The visible formulas include:

$$u = -\frac{1}{v^2} \frac{\partial u}{\partial t^2} + \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$$
$$v = \sqrt{\left(\frac{g\lambda}{2\pi} + \frac{2\pi\gamma}{\rho\lambda}\right)} \tan$$
$$= \int_{-\infty}^{\infty} (\alpha(k) e^{i(kx - \omega t)} + \beta(k) e^{i(kx + \omega t)}) dk$$

Semantics

Interpretation of the **meaning** symbols and operators in propositions.

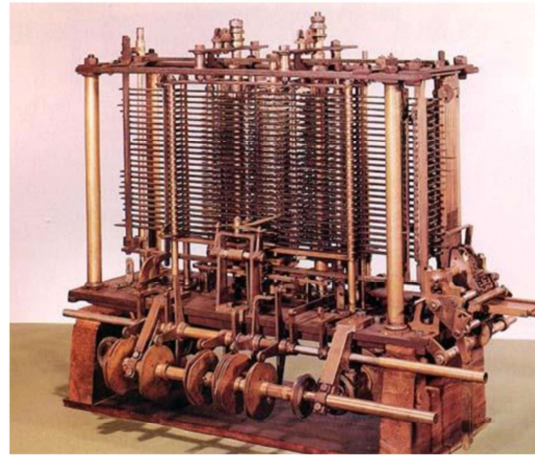


What do we need for a formal system? Firstly we need a specific notation, called a syntax, which is how we build well-formed formulae. Secondly, we want to give meaning to our formulae using semantics, which allows us to interpret symbols.

What We Need for a *Logic* or *Formal System*

Calculus

Processes for determining whether statement are true according to our semantics.



$$\text{LOGIC} = \text{SYNTAX} + \text{SEMANTICS} + \text{CALCULUS}$$

24

Lastly we need some sort of calculus, which is what allows us to manipulate these systems and determine which formulae are true.

Syntax, semantics, and calculus are necessary tools for our logic.

Logic Example

Programming Languages

```
# Naive Python code to compute ????  
n = 23  
f = 1  
  
for i in range(1,n+1):  
    f = f * i  
  
print ("The ??? of 23 is : ",end="")  
print (fact)
```

25

Let's apply these concepts to programming. Take a look at this simple function and think about how it's written and how its meaning is interpreted.

Logic Example

```
FUNCTION f(n:natural):natural;  
  BEGIN  
    IF n=0 THEN f:=1  
    ELSE f:=n*f(n-1);  
  END;
```

→ **Intended semantics**
Computing factorials

→ **Formal semantics**
 $f : n \mapsto n!$

→ **Procedural semantics**
What happens at
program execution

26

When writing an algorithm, the first important thing is to write code correctly in the program in which you're working, in this case Python. The syntax needs to be proper before we consider the meaning of our code.

Now we can consider three different types of semantics. The intended meaning is to calculate factorials, whether this is what really happens or not. The formal semantics, in this case the function f , will actually calculate factorials. The procedural semantics is how the program executes the function once we run the program.

Part 3: Propositional Logic

Intelligent Systems
Vrije Universiteit

27

Now we understand what we need for a formal logic system. One type of logic is called **propositional logic**, which we'll look at in this section.

Note that this topic will be extensively covered in the course Logic & Sets. In this course we introduce this concept so that you are able to understand its symbols and perform simple calculations.

Propositional Logic Example

Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$$\begin{array}{l} \neg P_{1,1} \\ \neg B_{1,1} \\ B_{2,1} \end{array} \quad \begin{array}{c} \wedge \\ \wedge \end{array}$$

"Pits cause breezes in adjacent squares"

$$\begin{array}{l} B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}) \\ B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}) \end{array}$$

"A square is breezy *if and only if* there is an adjacent pit"

28

How do we write correct logical formulae in which we can say something about truth? Propositional logic is a declarative language with distinct symbols that allow us to evaluate whether propositions are true or false. For example, the proposition $P_{i,j}$ is only true if there is a pit in $[i, j]$, which represents a rule of Wumpus World in propositional logic.

How do we write "Pits cause breezes in adjacent squares" in propositional logic? Unfortunately we must write every square to which this rule applies separately.

$B_{1,1}$ implies that there is a pit in $P_{1,2}$ or $P_{2,1}$, but also pit in $P_{1,2}$ or $P_{2,1}$ implies that $B_{1,1}$.

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

Propositional Logic

Syntax

Propositions: statements (symbols + operators)

Symbols: *things* in the universe, e.g. " $P_{1,2}$ "

Operators: *and, or, not, implies, equivalent*, e.g. " $P_{1,2}$ **or** $P_{2,1}$ "

Semantics

Propositions can be True or False, e.g. $S_{1,2}$, **thus** $W_{1,3}$ or $W_{2,2}$

Calculations

Next lecture

Let's break down the three parts of this formal system.

We have introduced propositions (statements), which are composed of symbols and operators. Operators are used to connect symbols with one another. We also have a semantics, which is used to evaluate whether each proposition is true or false. In the next lecture we will discuss calculations that we can perform with propositional logic.

Propositional Logic Syntax

The proposition symbols P_1, P_2 etc are sentences

If S is a sentence, $\neg S$ is a sentence (**negation**)

If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (**conjunction**)

If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (**disjunction**)

If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (**implication**)

If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (**biconditional**)

30

There are five operators in propositional logic: negation, conjunction, disjunction, implication, and the biconditional (also called double implication). We can use operators to connect sentences with one another. Sentences may have only one proposition but may also be complex.

The first rule stipulates that we can add a negation symbol to any existing sentence and form another well-formed sentence. The negation doesn't combine symbols, it simple acts on only one (*unary*). The following four operators connect symbols (*binary*) and say that we can use these operators to combine any two sentences and create a new, valid and well-formed sentence.

Propositional Logic Semantics

A sentence can either be True or False.

Mathematically an “*interpretation or validation function*” maps any proposition p to true (1) or false (0).

$$v(p) = 0 \text{ or } v(p) = 1$$

Simple Statements

$v(\text{warm_outside}) = 0$ means it is not warm outside

$v(B_{11}) = 1$ means B_{11} is true

We can demonstrate whether a proposition p is true or false using a validation function v . When v evaluates to 1 then p is true, when v evaluates to 0 then p is false.

Propositional Logic *Semantics*

Complex statements are compositions of simple statements. Their truth value is calculated from the meaning of its parts.

e.g. **Negation**

$\neg P$ is calculated as T/F from the meaning of P

$\neg P$ is only True when P is False

P	$\neg P$
1	0
0	1

Truth
Table

32

We can combine the simple statements we've seen using **operators** in order to make complex statements. One such operator is **negation** $\neg$, which translates to "not". The negation symbol means that the truth value of a statements changes from T $\rightarrow$ F or from F $\rightarrow$ T. e.g.

$$v(P) = 1$$

$$v(\neg P) = 0$$

$$v(\neg\neg P) = 1$$

Propositional Logic Semantics

Conjunction $\wedge$ and

$P \wedge Q$ is true *iff* P is true and Q is true

P	Q	$P \wedge Q$
1	1	1
1	0	0
0	1	0
0	0	0

Disjunction $\vee$ or

$P \vee Q$ is true *iff* P is true or Q is true.

At least one of P and Q are true.

P	Q	$P \vee Q$
1	1	1
1	0	1
0	1	1
0	0	0

33

The **conjunction** $\wedge$ operator is interpreted as “and”; a complex statement of two propositions joined by a conjunction is only true when both of these propositions are individually true, e.g.

If $v(P) = 1$ and $v(Q) = 1$, then $v(P \wedge Q) = 1$

If $v(P) = 0$ and $v(Q) = 1$, then $v(P \wedge Q) = 0$

If $v(P) = 0$ and $v(Q) = 0$, then $v(P \wedge Q) = 0$

The **disjunction** $\vee$ operator is interpreted as “or”; a complex statement of two propositions joined by a disjunction is true if either proposition is individually true, e.g.

If $v(P) = 1$ and $v(Q) = 1$, then $v(P \vee Q) = 1$

If $v(P) = 0$ and $v(Q) = 1$, then $v(P \vee Q) = 1$

If $v(P) = 0$ and $v(Q) = 0$, then $v(P \vee Q) = 0$

Propositional Logic Semantics

Implication $\rightarrow$

$P \rightarrow Q$ "P implies Q"

$P \rightarrow Q$ is True *iff* P is true and Q is true.

P	Q	$P \rightarrow Q$
1	1	1
1	0	0
0	1	1
0	0	1

$P \rightarrow Q$ is False when P is True and Q is False.

$$P \rightarrow Q = \neg P \vee Q$$

Rewrite the problem using disjunction when solving exercises.

34

The **implication** $\rightarrow$ operator is interpreted as "implies"; a complex statement of two propositions joined by an implication is only *false* when the first proposition is true and the second proposition is false, otherwise it is true, e.g.

If $v(P) = 1$ and $v(Q) = 0$, then $v(P \rightarrow Q) = 0$

Otherwise $v(P \rightarrow Q) = 1$

We can rewrite statements using an implication symbol by placing a $\neg$ in front of the first proposition and replacing $\rightarrow$ with $\vee$.

Propositional Logic Semantics

Double Implication $\Leftrightarrow$

$P \Leftrightarrow Q$ “P implies Q AND Q implies P”

$P \Leftrightarrow Q$ is true *iff* P and Q are both false or both true.

P	Q	$P \Leftrightarrow Q$
1	1	1
1	0	0
0	1	0
0	0	1

$$P \Leftrightarrow Q = (P \rightarrow Q) \wedge (Q \rightarrow P)$$

Rewrite the problem using conjunction when solving exercises.

35

The **double implication** $\Leftrightarrow$ operator is interpreted as “implies” going in both directions; a complex statement of two propositions joined by a conjunction is true when both propositions have the same truth value independently, e.g.

If $v(P) = 1$ and $v(Q) = 1$, then $v(P \Leftrightarrow Q) = 1$

If $v(P) = 0$ and $v(Q) = 0$, then $v(P \Leftrightarrow Q) = 1$

If $v(P) = 1$ and $v(Q) = 0$ (or vice versa), then $v(P \Leftrightarrow Q) = 0$

We can rewrite double implication statements by separating the propositions into two separate implications statements joined by a conjunction;

$$P \Leftrightarrow Q = (P \rightarrow Q) \wedge (Q \rightarrow P)$$

Recall from the previous slide that we can then rewrite this further using the rule $P \rightarrow Q = \neg P \vee Q$;

$$(P \rightarrow Q) \wedge (Q \rightarrow P) = (\neg P \vee Q) \wedge (\neg Q \vee P)$$

Semantics of Complex Sentences *Example 1*

e.g. $(p \rightarrow q) \wedge p$

Step 1: rewrite to $(\neg p \vee q) \wedge p$ because $p \rightarrow q = \neg p \vee q$

Step 2: write all variables in truth table

Step 3: Write substeps in column headers, make separate columns for negations $\neg$

Step 4: Start by filling in all combinations of T/F for the variables.

p	q	$\neg p$	$(\neg p \vee q)$	$(\neg p \vee q) \wedge p$
1	1	0	1	1
1	0	0	0	0
0	1	1	1	0
0	0	1	1	0

36

When presented with a complex sentence like $(p \rightarrow q) \wedge p$, we can best solve for its truth value by creating a **truth table** as seen on the right.

The *column headers* represent each variable in the sentence, in this case p and q , as well as the useful subsections of the sentence. The final column header should be the sentence in its entirety.

The amount of rows is equal to $(\# \text{ of variables})^2$, in this case $2^2 = 4$. If we had three variables, this would be $3^2 = 8$, and so on. We fill in the cells for the variable with 0s and 1s so that we have every combination of truth values for the variables. Here is an easy way to ensure that you have every combination:

In the first column, make the first half of the rows 1s and the second half 0s.

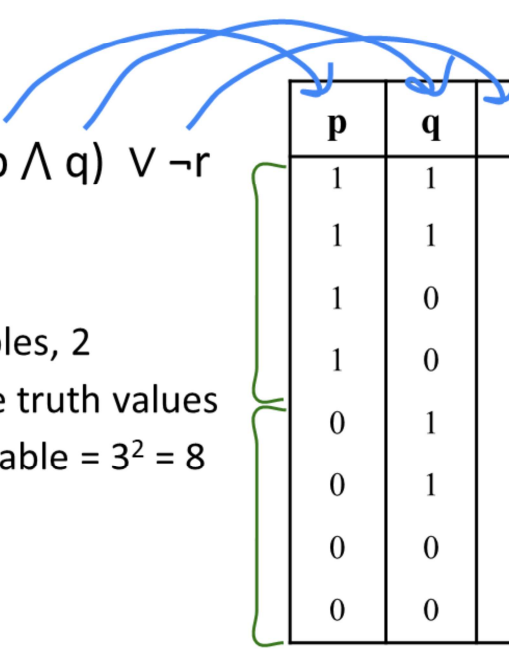
In the second column, alternate the 1s and 0s. If we have 3 variables, we alternate every other row (i.e. 1, 1, 0, 0, etc.)

Truth tables are useful because we can slowly build up to the entire sentence by using the truth values of its parts.

Complex Sentences *Example 2*

e.g. $(p \wedge q) \vee \neg r$

3 variables, 2
possible truth values
per variable = $2^3 = 8$
lines



p	q	r	$p \wedge q$	$\neg r$	$(p \wedge q) \vee \neg r$
1	1	1	1	0	1
1	1	0	1	1	1
1	0	1	0	0	0
1	0	0	0	1	1
0	1	1	0	0	0
0	1	0	0	1	1
0	0	1	0	0	0
0	0	0	0	1	1

This is a truth table for a statement with 3 variables. Notice how the 1s and 0s are filled in for the variables to ensure that we have every combination.

Complex Sentences *Practice Problems*

Assume that A, B, C, and D are all false. Find whether the following statements are true. Hint: you may not need to make a truth table.

1. $(A \wedge B) \vee \neg C \vee \neg D$
2. $(A \wedge B) \vee (\neg C \vee \neg D)$
3. $(A \rightarrow B) \vee (C \rightarrow D)$

Note: $\wedge$ means conjunction, $\vee$ means validation function.

Answers on next slide.

Complex Sentences Answers

$$\begin{aligned} 1. & (A \wedge B) \vee \neg C \vee \neg D \\ & (F \wedge F) \vee \neg F \vee \neg F \\ & (F) \vee T \vee T \end{aligned}$$

T

$$\begin{aligned} 1. & (A \wedge B) \vee (\neg C \vee \neg D) \\ & (F \wedge F) \vee (\neg F \vee \neg F) \\ & (F) \vee (T \vee T) \\ & F \vee T \end{aligned}$$

T

$$\begin{aligned} 3. & (A \rightarrow B) \vee (C \rightarrow D) \\ & (\neg A \vee B) \vee (\neg C \vee D) \\ & (T \vee F) \vee (T \vee F) \end{aligned}$$

T $\vee$ T

T

Knowledge Bases & Models

Recall that a **knowledge base** is a set of sentences that we assume to be true, written in curly bracket notation.

e.g. $\{\neg A, A \vee B, C\}$

Models are interpretation functions of the variables, or *validations*, which evaluate all formulas in a KB to true.

In other words, truth values that make a knowledge base true.

e.g. $v(A) = 0, v(B) = 1, v(C) = 1$ is model for $\{\neg A, A \vee B, C\}$

40

Recall our definition of a knowledge base KB , which is a collection of true statements written in set notation. **Models** are interpretations, or truth values, for our propositions that make the knowledge base true. There may be more than one model for every knowledge base, or there may be none

Let's take an easy example: if $KB = \{P, Q\}$, then we know P and Q must both be true in order to satisfy this knowledge base. Thus, the only model for this KB is $v(P) = 1, v(Q) = 1$.

Finding Models

How do we find a model for a knowledge base? [Look at which validations lead to the sentence \(KB\) being true.](#)

For the sentence $(p \wedge q) \vee \neg r$, we have the following models:

$v(p) = 1, v(q) = 1, v(r) = 1$

$v(p) = 1, v(q) = 1, v(r) = 0$

$v(p) = 1, v(q) = 0, v(r) = 0$

$v(p) = 0, v(q) = 1, v(r) = 0$

$v(p) = 0, v(q) = 0, v(r) = 0$

p	q	r	$p \wedge q$	$\neg r$	$(p \wedge q) \vee \neg r$
1	1	1	1	0	1
1	1	0	1	1	1
1	0	1	0	0	0
1	0	0	0	1	1
0	1	1	0	0	0
0	1	0	0	1	1
0	0	1	0	0	0
0	0	0	0	1	1

41

For complex statements, just looking at it is probably inadequate for finding a model. Instead, create a truth table and highlight the rows for which you have a 1 in the last column, in other words, where the statement is true. The truth values of the propositions in these rows are the *models* of this statement, or our *KB*.

Satisfiability, Validity, Entailment

A sentence is **satisfiable** if it is true in some interpretation, i.e. there exists a model.

e.g. $A \vee B, C$

A sentence is **unsatisfiable** if it is true in no interpretation, i.e. it has no model.

e.g. $A \wedge \neg A$

A sentence is **valid** if it is true in all interpretations, also called a **tautology**.

e.g. $A \vee \neg A$

A sentence is **entailed** by a KB K *iff* is true in all models of KB. We write $KB \models \alpha$.

e.g. $\{\neg A, A \vee B\} \models B$

42

Let's look at more useful definitions for describing knowledge bases and their models.

If we can find a model for a sentence, then it is **satisfiable**. Otherwise, it is **unsatisfiable** because there does not exist a model.

A sentence is **valid** if it is always true, meaning whichever truth values we come up with the sentence will *always* be true.

When looking at a knowledge base and its models, a sentence which is true in all the models is considered **entailed**. In the example provided, B must be true in order for a model to be true, so it is entailed.

Part 4: SAT Solving

Intelligent Systems
Vrije Universiteit

43

The process of figuring out whether a knowledge base is satisfiable, i.e. if there exists a model, is called SAT solving.

SAT Solving

Satisfiability solving is the task of finding one or more models of the KB.

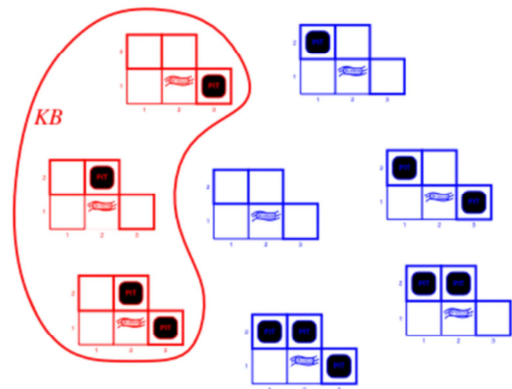
It is an old but still active field of research, used in hardware/software verification, planning & scheduling, control, protocol design, etc.

Example

In game scenarios the KB represents rules and game states. The models correspond to valid states in the game.

In red are models of Wumpus World.

$KB = \text{wumpus-world-rules} + \text{observations}$



An example of SAT solving is demonstrated using Wumpus World. In all the different states shown on the screen, only the states which agree with the observations and rules of the game are models.

Intelligent Systems Framework

Efficient, simple, correct, well- understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Progress in the intelligent systems framework.

Summary

Take Home Message

We can create formal logic systems using **syntax**, **semantics**, and **formulas** to manipulate symbols. **Truth tables** let us evaluate complex sentences.

We can apply **SAT Solving** to a **knowledge base** in order to find its **models**, used in applications such as games.

Thank you for your attention and have a lovely day.